Phase 1

RISC-V RV32I Assembly and Assembler

EEL 4768 Computer Architecture
Fall 2026

Learning Objectives

By the end of this phase you should be able to:

- Read and write RV32I assembly by hand, using only the base integer instruction set.
- Explain how a RISC-V instruction is encoded as a 32-bit word, and how that word is serialized to bytes in memory.
- Build a two-pass assembler (labels/symbol table, then encode) that handles the *entire* RV32I base ISA, not just the sample programs you were given.

Note

You may **only use instructions from the base RV32I ISA** in this phase – for both Part 1 and Part 2. See The RV32I reference card on Webcourses for the list of allowed instructions. Additionally, every program must terminate with `ecall` (see `examples/addition.s`), and your assembler should be able to compile this instruction. **Using any instruction outside base RV32I earns a zero on that problem**, even if the program otherwise produces the right answer.

1 Submission

- **One submission per group.** In this phase, you will manually submit your work to webcourses as a zip file. Students in multiple groups, or not in a group by the time of submission, will receive **no credit** for the submission.
- **Files to submit**:

```
gemm.s
mult.s
sobel.s
assembler.py
any additional files needed for your assembler
```

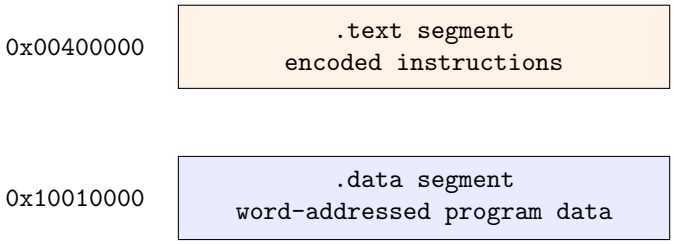
  Start each `.s` file from its skeleton in `phase_1/skeletons/` and `assembler.py` from `phase_1/skeletons/assembler.py`.

2 Grading Rubric

Category	Problem	Points
RISC-V Assembly	Multiplication	0.5
	GEMM	0.5
	Sobel	0.5
	subtotal	*1.5*
Assembler	Multiplication	0.166
	GEMM	0.166
	Sobel	0.166
	Instruction Test	1.0
	subtotal	*1.5*
Total		**3.0**

3 Memory Layout

This phase, and future phases, uses a memory layout where instructions start at `0x00400000` and the program's data starts at `0x10010000`.



Note

Your assembler must place `.data` at `0x10010000` and `.text` at `0x00400000` *exactly*. Any address arithmetic your encoded program relies on, such as branching, jumping, loading, and storing will compute with this assumption.

4 Part 1: RISC-V Assembly

Each problem provides a skeleton RISC-V assembly file (.s) with a pre-populated .data section containing an example input, a zeroed output, and a main: label. Write your program starting at main: and terminate using ecall, as demonstrated in addition.s.

Note

The numbers in your skeleton's `.data` section (e.g. `a: .word 22` in `mult.s`) are just *one* example configuration. Final grading will use multiple configurations design to stress test your program.

4.1 Multiplication

Multiply two words `a` and `b`, storing the result in `c`. The base RV32I ISA has no multiply instruction, so you must build one out of shifts and adds. You will be tasked with implementing a **bit-serial multiplication algorithm**. Each loop examines one bit of B at a time. When $B_i = 1$, a shifted value of A is accumulated ($A \ll i$). This can be formulated as:

$$C = A \times B, \qquad C = \sum_{i=0}^{N-1} B_i \, (A \ll i)$$

C Example

```
for(i = 0; i < 32; i++):
      C += B * (A << i)
```

4.2 General Matrix Multiplication (GEMM)

Compute the GEMM of two 4×4 matrices `a` and `b`, storing the 4×4 result in `c`:

$$\begin{bmatrix} C_{11} & \cdots & C_{1j} \\ \vdots & \ddots & \vdots \\ C_{i1} & \cdots & C_{ij} \end{bmatrix} = \begin{bmatrix} A_{11} & \cdots & A_{1k} \\ \vdots & \ddots & \vdots \\ A_{i1} & \cdots & A_{ik} \end{bmatrix} \times \begin{bmatrix} B_{11} & \cdots & B_{1j} \\ \vdots & \ddots & \vdots \\ B_{k1} & \cdots & B_{kj} \end{bmatrix}, \qquad C_{ij} = \sum_{k=0}^{K-1} A_{ik} \times B_{kj}$$

with $M{=}N{=}K{=}4$. Use exactly this loop ordering (m, then n, then k):

C Example

```
for (m = 0; m < M; m++) {
  for (n = 0; n < N; n++) {
    for (k = 0; k < K; k++) {
      C[m,n] += A[m,k] * B[k,n];
    }
  }
}
```

The skeleton stores each matrix as **16 consecutive words in row-major order** (row 0's four words, then row 1's, ...), not as a true 2D structure – there is no RV32I "2D array." You must convert a (row, col) index into a byte offset yourself.

4.3 Sobel Filter

The Sobel filter approximates the horizontal and vertical intensity gradient of an image by convolving it with two 3×3 kernels, G_x and G_y:

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \qquad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

Note

Use the kernel values exactly as given in your skeleton's `.data` section, not from memory of the formula above. The provided `sobel.s` skeleton (and the grading data) define `gy` as `[[1,2,1],[0,0,0],[-1,-2,-1]]`, which differs from the kernels above to more thoroughly test your convolution logic.

Convolving a 3×3 kernel over a 5×5 image with stride 1 produces a 3×3 output: at each of the 9 output positions, take the element-wise product of the kernel with the 3×3 window of the image under it, and sum the 9 products.

Breakdown

Given the 5×5 image A and kernel B below, and the top-left 3×3 window A_{w0}:

$$A = \begin{bmatrix} 4 & 5 & 1 & 2 & 3 \\ 3 & 0 & 1 & 3 & 5 \\ 4 & 6 & 7 & 3 & 1 \\ 2 & 1 & 0 & 8 & 9 \\ 3 & 6 & 6 & 3 & 4 \end{bmatrix} \qquad B = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} \qquad A_{w0} = \begin{bmatrix} 4 & 5 & 1 \\ 3 & 0 & 1 \\ 4 & 6 & 7 \end{bmatrix}$$

Element-wise product of A_{w0} and B, summed:

$$\begin{aligned} &(4)(-1) + (5)(-2) + (1)(-1) + (3)(0) + (0)(0) + (1)(0) + (4)(1) + (6)(2) + (7)(1) \\ &\quad = -4 - 10 - 1 + 0 + 0 + 0 + 4 + 12 + 7 = 8 \end{aligned}$$

so the first output element is 8. Slide the window one column right (stride 1) to get the next output element, wrapping to the next row after the last column, until all 9 output positions are filled. Do this independently for G_x and G_y, then add the two 3×3 results element-wise to get the final output:

$$C_{i,j} = \mathtt{gx}_{i,j} + \mathtt{gy}_{i,j}$$

A visualization of the sliding-window computation can be found here: convolution visualizer.

5 Part 2: RISC-V Assembler

You will implement an assembler for RV32I (including `ecall`) in Python, starting from `skeletons/assembler.py`. It takes one command-line argument, the path to a `.s` file, and must produce four output files (Section 5.2).

Note

Your assembler is graded on more than the three programs above. It will also include a brute-force test over all instructions from the reference card, as well as the addition.s example, accross multiple configurations.

5.1 Register Names

RV32I has 32 general-purpose registers. Any of them may appear by number or by its calling-convention (ABI) name in a program your assembler must accept:

Number	ABI name	Number	ABI name
x0	zero	x16	a6
x1	ra	x17	a7
x2	sp	x18	s2
x3	gp	x19	s3
x4	tp	x20	s4
x5	t0	x21	s5
x6	t1	x22	s6
x7	t2	x23	s7
x8	s0 (fp)	x24	s8
x9	s1	x25	s9
x10	a0	x26	s10
x11	a1	x27	s11
x12	a2	x28	t3
x13	a3	x29	t4
x14	a4	x30	t5
x15	a5	x31	t6

Table 1: RV32I integer registers, numeric and ABI names.

5.2 Output Files

For an input file `addition.s`, your assembler must produce four files, all one entry per line:

Output file	Contents	Line format
`addition.hex.txt`	instruction bytes, hex	`0xNN`, one byte per line
`addition.bin.txt`	instruction bytes, binary	8-bit binary, one byte per line
`addition_data.hex.txt`	data bytes, hex	`0xNN`, one byte per line
`addition_data.bin.txt`	data bytes, binary	8-bit binary, one byte per line

Breakdown

Byte order. Every file is one *byte* per line, in **little-endian** order (least-significant byte first). This applies to both the instruction words and multi-byte data words.
For example, `addition.s` begins with `lui t0, 0x10010`. Encoding a `lui` instruction packs the 20-bit immediate into bits [31:12], the destination register into bits [11:7], and the opcode `0110111` into bits [6:0]:

$$\underbrace{\texttt{0x10010}}_{\text{imm[31:12]}} \ll 12 \;\Big|\; \underbrace{5}_{\texttt{t0}=\texttt{x5}} \ll 7 \;\Big|\; \underbrace{\texttt{0x37}}_{\text{opcode}} = \texttt{0x100102B7}$$

As bytes, from address `0x00400000` upward, little-endian order means the *least significant* byte (`0xB7`) is written first:

```
0xb7   <- byte 0 (0x00400000): bits [7:0]
0x02   <- byte 1 (0x00400001): bits [15:8]
0x01   <- byte 2 (0x00400002): bits [23:16]
0x10   <- byte 3 (0x00400003): bits [31:24]
```

This matches the first four lines of the reference `examples/addition_sol.hex.txt`.

5.3 Memory Addresses

Similar to Section 3, your assembler must place:

- **Instructions** starting at `0x00400000`
- **Data** starting at `0x10010000`

Any label address you compute for a branch/jump target or a `.data` symbol must be computed against these bases.

6 Testing Your Work

The repository ships a self-check harness (`phase_1/README.md`, `phase_1/run_test.sh`, `phase_1/source_test/`) that runs a check of your four assembly programs under one data configuration, plus your assembler on `addition.s` and on a brute-force instruction test. The test reports `PASS`/`FAIL` and stores the output of your tests in `phase_1/output/`.

```
./phase_1/run_test.sh mycheck
...
PASS  Assembly:  gemm (config 1)
FAIL  Assembly:  mult (config 1)  --  result memory differs from the expected ...
...
5/6 PASS
```